**WEB PROGRAMMING**

**PERSONAL ASSIGNMENT – SEMESTER PROJECT**

**Individual Web Application Development Project**

**Weight:** 40% of the Final Course Grade
**Duration:** Approximately 8 Weeks
**Submission:** Individual
**Development Platform:** PHP + MySQL
**Project Type:** Dynamic, Database-Driven Web Application

---

## 1. PROJECT OVERVIEW

The Personal Assignment is an **individual semester-long web application development project** for the Web Programming course.

Each student is required to analyze, design, develop, test, document, and demonstrate a complete dynamic website based on **one assigned project topic**.

The purpose of this assignment is to assess the student's ability to integrate the concepts and technical skills covered throughout the course into a realistic web application.

The project must demonstrate the student's ability to:

- Analyze a real-world web application scenario.
- Identify users, roles, requirements, and business processes.
- Design an appropriate website structure and user interface.
- Develop responsive web pages using HTML5 and CSS3.
- Implement client-side interactions using JavaScript.
- Implement server-side functionality using PHP.
- Design and implement a relational database using MySQL.
- Retrieve, insert, update, and delete database records appropriately.
- Implement AJAX-based interactions.
- Implement user authentication and session management.
- Apply input validation and basic web security practices.

- Implement search and/or filtering functionality.
- Apply basic Search Engine Optimization (SEO) techniques.
- Implement business logic specific to the assigned project topic.
- Test and debug a complete web application.
- Explain technical and design decisions made during development.

The project must be developed **progressively throughout the project period**. Students are strongly discouraged from waiting until the final week to implement the entire system.

---

**2. PROJECT LEARNING OBJECTIVES**

Upon completion of the project, students should be able to:

**LO1 – Web Application Development**

**LO2 – Server-Side Programming**

**LO3 – Database Development**

**LO4 – Dynamic Web Interaction**

**LO5 – Web Security and SEO**

**LO6 – Problem Solving and Technical Communication**

---

**3. INDIVIDUAL PROJECT REQUIREMENT**

This is an **individual assignment**.

Each student must independently complete:

- Requirement analysis
- UI/UX design
- Database design
- Front-end development
- Back-end development
- Testing

- Documentation
- Final demonstration

Students may consult textbooks, official documentation, tutorials, and other learning resources.

However, each student must produce an **independent implementation**.

Students must not:

- Copy another student's project.
- Share source code with another student.
- Submit another student's work.
- Reuse a previous student's project with only superficial modifications.
- Download a complete project and submit it as their own work.

---

**4. PROJECT TOPIC ASSIGNMENT**

Each student will be assigned **ONE (1)** of the following six project topics.

| Topic ID | Project Topic |
|---|---|
| **T1** | Gym & Fitness Center Website |
| **T2** | Spa & Beauty Center Website |
| **T3** | Medical Clinic Website |
| **T4** | Dental Clinic Website |
| **T5** | Law Firm Website |
| **T6** | Short-Course Training Center Website |

The six topics are intentionally designed around different business processes.

Students are expected to implement the **business logic of their assigned topic**, not merely create a generic CRUD website.

**5. TOPIC ASSIGNMENT RULE**

To distribute project topics fairly and reduce duplication among students, the topic will be assigned based on the student's Student ID.

**Assignment Formula**

**Topic Number = StudentID mod 6**

| StudentID mod 6 | Assigned Topic |
|---|---|
| **0** | T1 – Gym & Fitness Center Website |
| **1** | T2 – Spa & Beauty Center Website |
| **2** | T3 – Medical Clinic Website |
| **3** | T4 – Dental Clinic Website |
| **4** | T5 – Law Firm Website |
| **5** | T6 – Short-Course Training Center Website |

**Example**

For Student ID 20260018:

20260018 mod 6 = 0

Therefore:

**Assigned Topic: T1 – Gym & Fitness Center Website**

Once assigned and approved, the project topic **may not be changed without explicit instructor approval**.

## 6. GENERAL SYSTEM REQUIREMENTS

Regardless of the assigned topic, every project must satisfy the following requirements.

---

### 6.1 Website Structure

The website must contain an appropriate:

- Header
- Logo or website identity
- Navigation menu
- Main content area
- Footer

The navigation structure must reflect the assigned project topic.

Students are encouraged to design their own navigation structure rather than copying a common template.

---

### 6.2 Responsive Design

The website must be responsive and usable on:

- Desktop computers
- Tablets
- Mobile devices

Students may use:

- Bootstrap, or
- Custom CSS with media queries.

The responsive implementation must be demonstrated during the final presentation.

---

### 6.3 Front-End Technologies

The project must use:

- HTML5
- CSS3
- JavaScript

Students may use appropriate front-end libraries or frameworks where permitted.

However, the student must be able to explain the purpose of the libraries used.

---

**6.4 Back-End Technologies**

The project must use:

- PHP
- MySQL

The application must demonstrate genuine server-side processing.

Examples include:

- Form processing
- Database operations
- Authentication
- Session management
- Business logic
- Search/filter processing
- Data validation

A website consisting primarily of static HTML pages does not satisfy the project requirements.

---

**6.5 Database Requirements**

The application must use a relational MySQL database.

The database must contain multiple related tables appropriate to the assigned topic.

The database design must:

- Use appropriate primary keys.
- Use foreign keys where appropriate.
- Represent meaningful relationships.
- Avoid unnecessary duplication.
- Store application data persistently.
- Use appropriate data types.
- Maintain reasonable data integrity.

At minimum, the database should contain:

- User/account information
- Main business entities
- Supporting/reference entities
- Transaction or activity records where appropriate

Students must submit an **ER diagram**.

The ER diagram must correspond to the actual database implemented in the project.

---

**6.6 CRUD Operations**

The project must demonstrate appropriate database operations.

Depending on the assigned topic, the system must implement relevant:

- Create
- Read
- Update
- Delete

operations.

Not every entity must necessarily support all four operations. The operations should reflect the actual business requirements.

For example:

- An administrator may create and update services.
- A customer may create an appointment.
- A staff member may update appointment status.
- A user may delete a favorite item.

---

### 6.7 User Authentication

The project must implement a basic authentication system.

At minimum:

**Registration**

Users should be able to create an account using appropriate information.

**Login**

Users should be able to authenticate using their credentials.

**Logout**

Users should be able to securely terminate their session.

**Password Security**

Passwords must **not** be stored as plain text.

Students must use an appropriate password hashing mechanism provided by PHP or their chosen framework/library.

**Session Management**

The system must use sessions appropriately to maintain authenticated users.

---

### 6.8 Authorization and User Roles

The application must implement appropriate access control.

Depending on the project topic, the system should contain at least two meaningful user roles where appropriate.

Examples:

- Administrator / Customer

- Staff / Customer
- Trainer / Member
- Doctor / Patient
- Lawyer / Client
- Instructor / Student

A user should not be able to access functionality that is restricted to another role.

For example:

A customer should not be able to access an administrator's service-management page simply by entering the URL manually.

---

**6.9 Search and Filtering**

The application must provide an appropriate mechanism for users to find relevant information.

Depending on the topic, this may include:

- Keyword search
- Category filtering
- Date filtering
- Price filtering
- Location filtering
- Service filtering
- Instructor/doctor/trainer filtering
- Availability filtering
- Status filtering

The search/filter functionality must operate on **database-driven data**.

---

**6.10 AJAX Requirements**

Each project must implement **at least TWO (2) meaningful AJAX features**.

AJAX functionality must provide a genuine dynamic interaction.

Examples include:

- Live search
- Dynamic filtering
- Loading available time slots
- Checking availability
- Updating booking status
- Loading dependent data
- Adding/removing favorites
- Updating records without a full page reload
- Dynamic appointment selection

The instructor may ask the student to demonstrate how the AJAX functionality works during the final demonstration.

---

**6.11 Form Validation**

Forms must include appropriate validation.

Students should implement:

**Client-side validation**

Using JavaScript where appropriate.

**Server-side validation**

Using PHP.

Server-side validation is mandatory because client-side validation alone is insufficient.

Examples:

- Required fields
- Email format
- Password requirements

- Numeric ranges
- Date validation
- Duplicate records
- Invalid selections

---

**6.12 Basic Web Security**

Students must demonstrate awareness of fundamental web security practices.

The project should address relevant issues including:

- SQL injection prevention
- Password hashing
- Server-side validation
- Output escaping
- Authentication
- Authorization
- Session management
- Protection of restricted functionality

Students must explain the security techniques used in their report.

---

**6.13 SEO Requirements**

Each project must implement at least **THREE (3)** basic SEO practices.

Possible techniques include:

- Meaningful page titles
- Meta descriptions
- Semantic HTML
- Appropriate heading structure
- Friendly URLs

- Image alt attributes
- Sitemap
- Meaningful internal links

The selected techniques must be appropriate for the project.

---

**6.14 Navigation and Breadcrumbs**

The website must provide clear navigation.

Where appropriate, the application should provide breadcrumb navigation.

For example:

**Home → Services → Personal Training → Service Details**

Breadcrumbs are expected particularly on pages with multiple levels of content.

---

**7. TOPIC-SPECIFIC PROJECT REQUIREMENTS**

The following requirements are **mandatory** for each assigned topic.

Students must implement both the common requirements in Section 6 and the topic-specific requirements in this section.

---

**T1 – GYM & FITNESS CENTER WEBSITE**

**7.1 Project Scenario**

Develop a dynamic website for a gym or fitness center.

The website should allow customers to explore membership plans, fitness classes, trainers, and class schedules.

The system should support the management of memberships and class bookings.

---

**7.2 Required Functionalities**

**A. Membership**

The system must provide:

- Membership plans
- Membership description
- Membership duration
- Membership price
- Membership status

Users should be able to view available membership plans.

---

**B. Fitness Classes**

The system must provide:

- Fitness class information
- Class category
- Class description
- Trainer
- Schedule
- Maximum capacity

---

**C. Trainers**

The website must provide trainer profiles containing appropriate information such as:

- Name
- Profile photo
- Specialization
- Short biography
- Classes taught

---

**D. Class Booking**

Authenticated members must be able to:

- View available classes.
- View class schedules.
- Book a class.
- View their bookings.
- Cancel a booking when allowed.

---

**7.3 Mandatory Business Logic**

The project must implement:

**Membership → Class → Schedule → Booking**

The system must enforce class capacity.

For example:

If a class has a capacity of 20 members, the system must not allow additional bookings after all 20 places have been reserved.

---

**7.4 Suggested Database Entities**

Students may design their own database, but it should appropriately represent entities such as:

- Users
- Members
- Membership Plans
- Trainers
- Fitness Classes
- Class Schedules
- Bookings

---

**T2 – SPA & BEAUTY CENTER WEBSITE**

**8.1 Project Scenario**

Develop a dynamic website for a spa or beauty center.

Customers should be able to explore services, beauty specialists, prices, service duration, and appointment availability.

---

**8.2 Required Functionalities**

**A. Services**

The system must provide:

- Service categories
- Service names
- Descriptions
- Prices
- Service duration

---

**B. Beauty Specialists**

The system must provide staff profiles including:

- Name
- Position or specialization
- Profile photo
- Short biography
- Services offered

---

**C. Appointment Booking**

Customers must be able to:

- Select a service.
- Select an available staff member where applicable.
- Select a date/time.

- Submit an appointment.
- View appointment history.
- Cancel an appointment when allowed.

---

**8.3 Mandatory Business Logic**

The project must implement:

**Service → Staff → Date/Time → Appointment**

The system must prevent inappropriate double booking of the same staff member for overlapping appointments.

---

**8.4 Suggested Database Entities**

Possible entities include:

- Users
- Customers
- Services
- Service Categories
- Staff
- Staff Schedules
- Appointments

---

**T3 – MEDICAL CLINIC WEBSITE**

**9.1 Project Scenario**

Develop a dynamic website for a medical clinic.

Patients should be able to learn about doctors and medical services and request appointments.

---

**9.2 Required Functionalities**

**A. Doctors**

The website must provide:

- Doctor profiles
- Medical specialty
- Biography
- Areas of expertise
- Consultation information

---

**B. Medical Services**

The website must provide:

- Service name
- Description
- Category
- Fee or price information where appropriate
- Estimated consultation duration where appropriate

---

**C. Doctor Schedules**

The system must provide appropriate information about:

- Available doctors
- Working dates
- Working hours
- Appointment availability

---

**D. Appointment**

Patients must be able to:

- Select a doctor.

- Select a service.
- Select an available date/time.
- Submit an appointment.
- View appointment history.
- Cancel an appointment when allowed.

---

**9.3 Mandatory Business Logic**

The project must implement:

**Patient → Doctor → Schedule → Appointment**

The system must prevent two patients from booking the same doctor and time slot.

---

**9.4 Important Data Requirement**

Students must use **fictional/sample data only**.

Students must not collect or use real patients' medical information for this academic project.

---

**9.5 Suggested Database Entities**

Possible entities include:

- Users
- Patients
- Doctors
- Specialties
- Medical Services
- Doctor Schedules
- Appointments

---

## T4 – DENTAL CLINIC WEBSITE

### 10.1 Project Scenario

Develop a dynamic website for a dental clinic.

Patients should be able to explore dental services, dentists, treatment information, schedules, and appointments.

---

### 10.2 Required Functionalities

#### A. Dentists

The website must provide:

- Dentist profiles
- Dental specialty
- Biography
- Areas of expertise
- Working schedule

---

#### B. Dental Services

The website must provide:

- Service categories
- Service description
- Price information
- Estimated duration where appropriate

Examples:

- General dental examination
- Teeth cleaning
- Dental filling
- Teeth whitening

- Orthodontic consultation
- Dental implant consultation

---

**C. Appointment**

Patients must be able to:

- Select a dentist.
- Select a dental service.
- Select an available time.
- Submit an appointment.
- View appointment history.
- Cancel an appointment where permitted.

---

**10.3 Mandatory Business Logic**

The system must implement:

**Patient → Dentist → Dental Service → Appointment**

The system should account for service duration when determining appointment availability.

For example:

A 60-minute treatment should not be scheduled into a time slot that is only available for 30 minutes.

---

**10.4 Suggested Database Entities**

Possible entities include:

- Users
- Patients
- Dentists
- Dental Services

- Dental Categories
- Dentist Schedules
- Appointments
- Treatment Records

All patient and treatment data used for demonstration must be **fictional sample data**.

---

**T5 – LAW FIRM WEBSITE**

**11.1 Project Scenario**

Develop a dynamic website for a law firm.

The website should allow visitors to explore the firm's legal services, practice areas, lawyers, and consultation options.

The project should focus on **legal consultation and case/request management**, rather than online legal transactions.

---

**11.2 Required Functionalities**

**A. Legal Services**

The website must provide:

- Legal service categories
- Practice areas
- Service descriptions
- Related lawyers
- Relevant information for potential clients

Examples:

- Corporate law
- Labor law
- Civil law
- Family law

- Intellectual property
- Real estate law

---

**B. Lawyer Profiles**

The website must provide:

- Lawyer name
- Profile photo
- Position
- Practice areas
- Professional biography
- Areas of expertise

---

**C. Consultation Request**

Authenticated users should be able to submit a consultation request containing appropriate information such as:

- Legal service
- Brief description of the issue
- Preferred contact method
- Preferred consultation time

---

**11.3 Mandatory Business Logic**

The system must implement:

**Client → Legal Service → Lawyer → Consultation Request**

The consultation request must have a status workflow.

At minimum:

**Submitted → Under Review → Accepted → Completed**

The system should allow an authorized staff/lawyer account to update the status.

---

### 11.4 Suggested Database Entities

Possible entities include:

- Users
- Clients
- Lawyers
- Practice Areas
- Legal Services
- Consultation Requests
- Request Status

Students should use fictional case information only.

---

## T6 – SHORT-COURSE TRAINING CENTER WEBSITE

### 12.1 Project Scenario

Develop a dynamic website for a training center that offers short-term professional or technical courses.

Possible courses include:

- Web Development
- Python Programming
- Data Analysis
- Cybersecurity
- Digital Marketing
- Graphic Design
- Office Productivity

---

### 12.2 Required Functionalities

#### A. Course Catalog

The website must provide:

- Course name
- Course category
- Description
- Duration
- Tuition fee
- Learning objectives
- Course status

---

#### B. Instructor Profiles

The website must provide:

- Instructor name
- Profile photo
- Professional background
- Area of expertise
- Courses taught

---

#### C. Class Schedule

The system must support course schedules containing information such as:

- Course
- Instructor
- Start date
- End date
- Class days

- Class time
- Maximum number of students
- Enrollment status

---

**D. Course Enrollment**

Students/users must be able to:

- View available courses.
- View class schedules.
- Select a class.
- Enroll in a class.
- View enrollment history.
- Cancel enrollment where appropriate.

---

**12.3 Mandatory Business Logic**

The system must implement:

**Course → Class Schedule → Instructor → Enrollment**

The system must enforce class capacity.

For example:

If a class has a maximum capacity of 25 students, the system must prevent additional enrollment once the capacity has been reached.

---

**12.4 Suggested Database Entities**

Possible entities include:

- Users
- Students
- Courses

- Course Categories
- Instructors
- Class Schedules
- Enrollments

---

**13. AJAX REQUIREMENTS BY TOPIC**

To ensure that AJAX is meaningful rather than artificially added, students should implement AJAX in functions relevant to their topic.

| Topic | Examples of Appropriate AJAX Features |
|---|---|
| **T1 – Gym** | Class search/filtering, available seats, booking/cancellation |
| **T2 – Spa** | Service filtering, available staff/time slots, appointment booking |
| **T3 – Medical Clinic** | Doctor filtering, available appointment slots, appointment status |
| **T4 – Dental Clinic** | Dentist/service filtering, available time slots, appointment booking |
| **T5 – Law Firm** | Lawyer/service filtering, consultation request status |
| **T6 – Training Center** | Course filtering, class availability, enrollment |

The examples above are suggestions. Students may implement other meaningful AJAX functionality with instructor approval.

---

**14. REQUIRED WEBSITE PAGES / FUNCTIONAL AREAS**

The exact number of pages will depend on the selected topic.

However, every project must contain appropriate equivalents of the following functional areas:

**Public Area**

- Home
- About / Organization Information

- Main service/content page
- Search/filter page
- Detail pages
- Contact page

**Authentication Area**

- Register
- Login
- Logout
- User profile

**User Area**

A user dashboard or equivalent functionality appropriate to the project.

Examples:

- My Bookings
- My Appointments
- My Enrollments
- My Consultation Requests
- My Membership

**Management Area**

An appropriate administrative or staff management area.

Examples:

- Manage users
- Manage services
- Manage schedules
- Manage bookings
- Manage appointments
- Manage courses

- Manage requests

The exact pages may vary according to the assigned topic.

---

**15. USER INTERFACE REQUIREMENTS**

The website must demonstrate a coherent visual design.

Students should pay attention to:

- Consistent typography
- Consistent spacing
- Color scheme
- Button design
- Form design
- Navigation
- Responsive layout
- Appropriate use of images
- Readability
- Accessibility considerations

The website should look like a coherent application rather than a collection of unrelated pages.

---

**16. DATABASE DESIGN REQUIREMENTS**

Students must submit an ER diagram showing the main entities and relationships.

The database should demonstrate appropriate use of:

**Primary Keys**

Each major entity must have an appropriate primary key.

**Foreign Keys**

Relationships between entities should be represented using foreign keys where appropriate.

**One-to-Many Relationships**

Examples:

- One trainer → many classes
- One doctor → many appointments
- One course → many class schedules

**Many-to-Many Relationships**

Where appropriate, students should use an intermediate/junction table.

For example:

Students ↔ Courses

may be represented through:

Enrollments

The database design must be consistent with the actual application.

---

**17. TESTING REQUIREMENTS**

Students must test their application before submission.

Testing should cover at least:

**Functional Testing**

- Registration
- Login
- Logout
- CRUD operations
- Search
- Filtering
- Booking/appointment/enrollment
- AJAX functionality

**Validation Testing**

Students should test:

- Empty fields
- Invalid input
- Invalid email
- Invalid dates
- Duplicate records
- Invalid selections

**Authorization Testing**

Students should verify that:

- Unauthorized users cannot access restricted pages.
- Users cannot perform actions belonging to another role.

**Responsive Testing**

Students should test:

- Desktop
- Tablet
- Mobile

---

**18. PROJECT DELIVERABLES**

Each student must submit **all** of the following.

---

**18.1 Source Code**

The complete project source code.

The project should have a clear and logical structure.

Students must not submit unnecessary files such as:

- Temporary files
- Cache files

- IDE configuration files unless necessary
- Large unused assets

---

**18.2 Database SQL File**

The SQL file must include:

- Database/table creation
- Primary keys
- Foreign keys where applicable
- Sample data

The instructor should be able to recreate the database from the submitted SQL file.

---

**18.3 ER Diagram**

The ER diagram may be submitted as:

- PDF
- PNG
- JPG

The diagram must correspond to the actual database.

---

**19. PROJECT REPORT**

Each student must submit a **Project Report in PDF format**.

**Recommended length:**

**8–12 pages**, excluding cover page and appendices.

The report should include:

**Chapter 1 – Introduction**

- Project title
- Background

- Problem statement
- Project objectives

**Chapter 2 – System Requirements**

- Target users
- User roles
- Functional requirements
- Non-functional requirements

**Chapter 3 – System Design**

- System architecture
- Website structure
- Navigation structure
- UI design decisions

**Chapter 4 – Database Design**

- Database overview
- ER diagram
- Major entities
- Relationships
- Explanation of important tables

**Chapter 5 – Implementation**

Explain the implementation of:

- PHP
- MySQL
- Authentication
- AJAX
- Search/filtering
- Business logic

- Responsive design

**Chapter 6 – Security and SEO**

Explain:

- Input validation
- Password security
- SQL injection prevention
- Authorization
- SEO techniques

**Chapter 7 – Testing**

Include:

- Test cases
- Expected results
- Actual results
- Important issues discovered and fixed

**Chapter 8 – Challenges and Lessons Learned**

Discuss:

- Major technical challenges
- Solutions
- Lessons learned

**Chapter 9 – Conclusion**

Summarize:

- Achievements
- Limitations
- Possible future improvements

---

**20. FINAL DEMONSTRATION**

Each student must individually demonstrate the completed application.

The demonstration must show:

1. Home page
2. Main functionality
3. Database-driven functionality
4. Authentication
5. Search/filtering
6. AJAX features
7. Topic-specific business logic
8. Management functionality
9. Responsive design
10. Security-related implementation where appropriate

The instructor may ask questions about the implementation.

The student may also be asked to make a **small modification** to the application during the demonstration.

Examples:

- Modify a PHP function.
- Change a database field.
- Add a validation rule.
- Modify a business rule.
- Change a query.
- Add a simple UI element.

---

**21. GRADING RUBRIC**

The project is graded on a **100-point scale** and contributes **40% of the final course grade**.

| No. | Assessment Component | Description | Points |
|---|---|---|---|
| 1 | **UI Design & Navigation** | Structure, usability, consistency, navigation, visual design | **8** |
| 2 | **Responsive Design** | Desktop, tablet, and mobile support | **7** |
| 3 | **Database Design** | Tables, relationships, keys, data integrity, ER diagram | **10** |
| 4 | **PHP & Dynamic Functionality** | Server-side processing and PHP/MySQL integration | **10** |
| 5 | **AJAX Implementation** | At least two meaningful AJAX features | **10** |
| 6 | **Core Topic Functionality** | Required features for the assigned topic | **15** |
| 7 | **Topic-Specific Business Logic** | Correct implementation of the assigned topic's core workflow | **15** |
| 8 | **Authentication & Security** | Authentication, authorization, validation, password security | **8** |
| 9 | **Search & Filtering** | Database-driven search/filter functionality | **5** |
| 10 | **SEO** | At least three appropriate SEO practices | **3** |
| 11 | **Project Report** | Technical documentation, analysis, explanation, testing | **5** |
| 12 | **Final Demonstration** | Functionality, explanation, and understanding of implementation | **4** |
| | **TOTAL** | | **100** |

**Final Course Contribution**

The project score is converted to **40% of the final course grade**.

For example:

**Project Score = 85/100**

**Contribution = 85 × 40% = 34/40**

---

**22. GRADING PERFORMANCE LEVELS**

The following general descriptors will be used to interpret the project score.

| Score Range | General Performance Description |
|---|---|
| **90–100** | Excellent implementation with strong functionality, well-designed database, effective business logic, good usability, and clear technical understanding |
| **80–89** | Very good implementation with most requirements successfully satisfied and only minor weaknesses |
| **70–79** | Good implementation with the major requirements satisfied but with noticeable limitations |
| **60–69** | Satisfactory implementation with several incomplete or weak areas |
| **50–59** | Minimum acceptable implementation with significant limitations |
| **Below 50** | Insufficient implementation and/or major project requirements not satisfied |

These descriptors are intended to provide general guidance. The final score will be determined using the detailed rubric above.

---

**23. ACADEMIC INTEGRITY**

This is an **individual project**.

Students may use:

- Course materials
- Textbooks
- Official documentation
- Online tutorials

- Programming references
- Open-source libraries
- Framework documentation
- AI-assisted development tools, subject to the instructor's course policy

However, the student must understand and be able to explain the submitted implementation.

The following are prohibited:

- Copying another student's project.
- Sharing substantial source code.
- Submitting another student's work.
- Submitting a project obtained from the Internet as one's own.
- Reusing a previous semester's project with only superficial changes.
- Submitting substantially similar projects.

Projects that are substantially similar to another student's project may receive a **zero score**, depending on the severity of the violation.

---

**24. AI USAGE POLICY**

AI tools may be used as **learning and development assistants**, but they must not replace the student's own learning and technical understanding.

Students may use AI tools for purposes such as:

- Understanding programming concepts
- Explaining error messages
- Generating examples for learning
- Suggesting possible implementation approaches
- Improving wording or documentation
- Reviewing code
- Debugging assistance

Students remain fully responsible for:

- The correctness of the submitted code
- The originality of the project
- Understanding the implementation
- Database design
- Security
- Testing
- Final results

Students must not submit AI-generated code that they cannot explain or modify.

During the final demonstration, the instructor may ask students to explain or modify parts of their implementation to verify individual understanding.

---

**25. DEVELOPMENT SCHEDULE**

The project should be developed progressively over approximately **8 weeks**.

| Period | Recommended Activities |
|---|---|
| **Week 1** | Understand assigned topic, analyze requirements, identify users and roles |
| **Week 2** | Design database, ER diagram, system structure, wireframes |
| **Week 3** | Develop HTML/CSS structure and responsive layout |
| **Week 4** | Implement PHP/MySQL integration and CRUD functionality |
| **Week 5** | Implement authentication, authorization, and core business features |
| **Week 6** | Implement AJAX, search, and filtering |
| **Week 7** | Implement security, SEO, testing, and topic-specific business logic |
| **Week 8** | Final testing, documentation, debugging, and demonstration preparation |

The instructor may establish intermediate checkpoints during the semester.

---

**26. IMPORTANT PROJECT RULES**

Students must comply with all of the following:

1. This is an **individual project**.
2. Each student will receive **one assigned topic**.
3. Topic changes require instructor approval.
4. The project must use **PHP and MySQL**.
5. The website must be responsive.
6. The website must be database-driven.
7. Authentication is mandatory.
8. Appropriate authorization is mandatory.
9. At least **two meaningful AJAX features** are mandatory.
10. Search and/or filtering is mandatory.
11. At least **three SEO practices** are mandatory.
12. Basic web security practices are mandatory.
13. An ER diagram is mandatory.
14. A project report is mandatory.
15. A final individual demonstration is mandatory.
16. The assigned topic's **business logic is mandatory**.
17. All demonstration data must be appropriate for an academic project.
18. Medical and legal topics must use **fictional/sample data only**.
19. Students must be able to explain their own source code.
20. Students are responsible for ensuring that their submission is their own work.

## 27. MINIMUM REQUIREMENT CHECKLIST

Before submission, students should verify the following:

**Front-End**

- HTML5 implemented
- CSS3 implemented
- JavaScript implemented
- Responsive design implemented
- Consistent UI design
- Navigation implemented

**Back-End**

- PHP implemented
- MySQL implemented
- Database-driven content implemented
- CRUD operations implemented where appropriate
- Form processing implemented
- Server-side validation implemented

**Authentication**

- Registration
- Login
- Logout
- Password hashing
- Session management
- Authorization / role-based access where applicable

**Dynamic Interaction**

- Search/filtering
- AJAX Feature 1

- AJAX Feature 2

**Security**

- SQL injection protection
- Input validation
- Output escaping where appropriate
- Authentication protection
- Authorization protection

**SEO**

- Page titles
- Meta descriptions
- Semantic HTML
- Appropriate heading structure
- Other SEO practice(s)

**Database**

- Primary keys
- Foreign keys
- Appropriate relationships
- ER diagram
- SQL file
- Sample data

**Documentation**

- Project report
- Testing section
- Challenges and lessons learned
- Screenshots where appropriate

**Final Demonstration**

- Website works correctly
- Login demonstrated
- Database functionality demonstrated
- AJAX demonstrated
- Business logic demonstrated
- Responsive design demonstrated
- Student can explain the implementation

---

**28. FINAL NOTE TO STUDENTS**

The purpose of this assignment is **not to build the largest or most complicated website**.

The primary objective is to demonstrate that you understand how a dynamic web application is designed and developed from **front-end to back-end**.

A successful project should demonstrate:

**Good analysis → Good design → Good database → Good PHP implementation → Meaningful business logic → Good testing → Clear explanation**

Students should focus on **correctness, usability, code quality, database design, security, and understanding**, rather than unnecessary complexity.

A smaller but well-designed and fully functional application is preferable to a large application containing incomplete or poorly understood features.